Consultório VR — Guia final

Servidor dedicado no Linux (DCC) + Quest como cliente · Unity 6000.0.39f1 · repositório `danielreissss/Projeto-VR`

```
[Quest com APK] --Ethernet--> [Roteador 5G] --5G--> [Core 5G] --cabo--> [servidorNetuno 172.30.22.31: ConsultorioServer, UDP 7777]
```

Situação

Etapa	Onde	Estado
Build do servidor Linux (OpenXR desmarcado)	PC de casa	**Feito**
`ConsultorioServer.zip` publicado no Release `servidor-v1`	GitHub	**Feito**
Gerar o APK e instalar nos óculos	PC de casa	Falta (Parte 1)
Baixar e rodar o servidor	Servidor Linux, no lab	Falta (Parte 2)
Conectar o Quest e entrar na sala	Lab	Falta (Parte 3)

Como funciona: o servidor roda o cenário sem gráficos e é o dono da sala (porta UDP 7777, sem IP para configurar). O Quest roda o APK, conecta no IP do servidor e, se a conexão falhar ou cair, tenta de novo a cada 3 s. **Ordem:** servidor rodando, depois abrir o app nos óculos.

Parte 1 — PC de casa: APK e instalação nos óculos

1.1 Voltar a plataforma para Android

1. No Unity: **File → Build Profiles → Android → Switch Platform** (demora, reimporta os assets).
2. **Edit → Project Settings → XR Plug-in Management**, aba do **Android** (robô): confira que **OpenXR** está marcado. Só o OpenXR do PC/servidor foi desmarcado.

1.2 Conferir o IP do servidor na cena

1. Abra `Assets/Scenes/ConsultorioFinal.unity`.
2. Na Hierarchy, busque `t:LanDirectConnect` e selecione o objeto.
3. **Host Ip Address** = `172.30.22.31`, **Port** = `7777`. Salve com Ctrl+S.

1.3 Gerar o APK

Menu **Build → APK Meta Quest (Android)**. Saída: `E:\testeMultiplayer3.0\Builds\Android\ConsultorioVR.apk`.

1.4 Instalar no Quest (modo desenvolvedor ativo)

Quest no USB; no headset, aceite *Permitir depuração USB*. No PowerShell:

```
cd E:\testeMultiplayer3.0
adb devices
adb install -r Builds\Android\ConsultorioVR.apk
adb tcpip 5555
```

`adb devices` deve listar o aparelho como `device`. Repita para cada óculos (o mesmo APK serve para todos). O app aparece em **Biblioteca → Fontes desconhecidas → Consultorio VR**.

> Sem o `adb` no PATH, use o do Unity: `C:\Program Files\Unity\Hub\Editor\6000.0.39f1\Editor\Data\PlaybackEngines\AndroidPlayer\SDK\platform-tools\adb.exe`

1.5 Não leve a mudança do OpenXR para o GitHub

Desmarcar o OpenXR no PC alterou arquivos de configuração. Se for dar commit em algo, confira antes:

```
git status
```

Se aparecerem `Assets/XR/XRGeneralSettingsPerBuildTarget.asset` ou `Assets/XR/Settings/OpenXR Package Settings.asset`, não os inclua. Para voltar ao estado do GitHub (e religar o OpenXR do PC, útil para o Play com Meta Link):

```
git restore "Assets/XR/XRGeneralSettingsPerBuildTarget.asset" "Assets/XR/Settings/OpenXR Package Settings.asset"
```

> Faça o `git restore` **depois** de gerar todos os builds. Se religar o OpenXR do PC, o build do servidor volta a falhar.

Parte 2 — Lab: baixar e rodar o servidor

2.1 Entrar no servidor

Use o computador do lab, ligado ao cabo da rede DCC (de casa o `172.30.22.31` não é alcançável):

```
ssh admin@172.30.22.31
```

2.2 Baixar o servidor do Release

```
sudo apt install -y unzip tmux
mkdir -p ~/consultorio-build
cd ~/consultorio-build
wget https://github.com/danielreissss/Projeto-VR/releases/download/servidor-v1/ConsultorioServer.zip
unzip ConsultorioServer.zip
chmod +x ConsultorioServer.x86_64
ls
```

O `ls` deve mostrar `ConsultorioServer.x86_64` e a pasta `ConsultorioServer_Data`.

Se o `ls ConsultorioServer_Data` mostrar arquivos com `\` no nome (o zip do Windows usa barras invertidas), extraia assim:

```
sudo apt install -y libarchive-tools
rm -rf ConsultorioServer_Data
bsdtar -xf ConsultorioServer.zip
chmod +x ConsultorioServer.x86_64
```

2.3 Liberar a porta e anotar os IPs

```
sudo ufw status
sudo ufw allow 7777/udp
ip -4 addr
```

Anote os IPs do `ip -4 addr`. Se o IP que o roteador 5G enxerga não for `172.30.22.31`, use o passo 3.2.

2.4 Rodar

```
tmux new -s vr
cd ~/consultorio-build
./ConsultorioServer.x86_64 -logFile -
```

Pronto quando aparecer:

```
--- LAN: Iniciando como SERVIDOR DEDICADO na porta 7777 ---
```

Ação	Como
Sair sem parar o servidor	**Ctrl+B**, solte, **D**
Voltar ao servidor	`tmux attach -t vr`
Parar	**Ctrl+C** dentro do tmux
Conferir a porta (outro terminal)	`ss -ulpn | grep 7777`

Avisos de XR, OVR, áudio e GPU no log são esperados: o servidor não tem óculos nem placa de vídeo.

Parte 3 — Lab: conectar os óculos

3.1 Entrar na sala

1. Adaptador USB-C → Ethernet no Quest (de preferência com entrada de carga) e cabo no **roteador 5G**.
2. No Quest: **Configurações → Wi-Fi → desligar**.
3. Com o servidor rodando, abra **Biblioteca → Fontes desconhecidas → Consultorio VR**.

O app conecta sozinho e continua tentando até o servidor responder. Cada óculos que abrir o app entra na mesma sala.

3.2 Se o IP do servidor for outro

Não precisa refazer o APK. Pelo computador do lab com `adb`, com o Quest no USB (ou `adb connect IP_DO_QUEST:5555`):

```
"NOVO_IP" | Out-File -Encoding ascii server_ip.txt
adb push server_ip.txt /sdcard/Android/data/com.Danie.ConsultorioVR.DriveE/files/server_ip.txt
```

Para voltar ao IP da cena:

```
adb shell rm /sdcard/Android/data/com.Danie.ConsultorioVR.DriveE/files/server_ip.txt
```

3.3 Confirmar que o 5G está no caminho

De um PC ligado ao roteador 5G:

```
tracert 172.30.22.31
```

Latência abaixo de 1 ms indica rede local, sem o 5G no caminho. Pergunte ao professor em que ponto o servidor está ligado.

Parte 4 — Conferir no primeiro teste

- O log do servidor mostra a conexão do cliente e você aparece no cenário.
- **Avatar fantasma:** antes de qualquer Quest conectar, veja se o log já mostra um avatar criado (dono `0`), parado na origem. É o servidor criando um avatar para si mesmo. Se acontecer, me avise: a correção é pequena e só no servidor.
- Todos os jogadores recebem o mesmo avatar e nascem na posição (0,0,0). É o esperado nesta cena.
- Log do Quest: `adb logcat -s Unity | findstr LAN`

Parte 5 — Atualizar o servidor depois de mudar o projeto

1. Gere o servidor de novo (**Build → Servidor Linux (Dedicated Server)**, com OpenXR desmarcado) e o APK (Parte 1). Gere os dois a partir do **mesmo** projeto, senão os óculos não entram.
2. Compacte e publique um release novo (`servidor-v2`):

   ```
   Compress-Archive -Path E:\testeMultiplayer3.0\Builds\LinuxServer\* -DestinationPath E:\ConsultorioServer.zip -Force
   ```

3. No servidor, pare o servidor antigo (Ctrl+C), apague a pasta e repita o passo 2.2 com a tag nova.

Problemas comuns

Sintoma	Solução
Build do servidor: *OpenXR Build Failed*	**Project Settings → XR Plug-in Management**, aba PC/servidor: desmarcar **OpenXR** e **Initialize XR on Startup**.
`NullReferenceException` em `ScreenSpaceAmbientOcclusion` no Console	Só acontece com a plataforma Linux Server ativa. Não afeta o build; some ao voltar para Android.
`adb devices` vazio ou *unauthorized*	Aceite *Permitir depuração USB* no headset; use cabo de dados; modo desenvolvedor ativo.
`wget` dá 404	Confira a tag do release (`servidor-v1`) e se o release foi publicado (não é rascunho).
`Permission denied` ao rodar	`chmod +x ConsultorioServer.x86_64`
Servidor parou ao fechar o SSH	Rodar dentro do `tmux` (passo 2.4).
Óculos abrem o cenário mas ninguém aparece	Servidor rodando? Porta 7777/udp liberada? IP certo (passo 3.2)? Quest no cabo, Wi-Fi desligado?
`DllNotFoundException` ou `MetaMovementSDK` no log do servidor	Me envie o log.
Sem voz	Quest: Configurações → Apps → Consultorio VR → permitir Microfone.

Checklist

Em casa

- ☐ Plataforma Android, OpenXR marcado na aba Android
- ☐ IP `172.30.22.31` conferido no Lan Direct Connect
- ☐ APK gerado e instalado em todos os óculos
- ☐ Release `servidor-v1` publicado com o zip
- ☐ Adaptador USB-C → Ethernet (com carga) e cabo

No lab

- ☐ SSH ok; `ip -4 addr` anotado
- ☐ Servidor baixado, extraído e com `chmod +x`
- ☐ Porta 7777/udp liberada
- ☐ Servidor rodando no tmux com "SERVIDOR DEDICADO" no log
- ☐ Quest no cabo do roteador 5G, Wi-Fi desligado, app aberto
- ☐ Cliente aparece no log do servidor